K501 AIONARC Pipeline V3 — Überblick

Zeitanker: 1791997611 · 14. Oktober 2026

K501 GO. Wir haben mit Version 3 einen wichtigen Entwicklungsschritt erreicht: Aus der bisherigen Pipeline zur Beobachtung und Zustandsprotokollierung ist ein erster eigenständiger Kandidat für eine strukturelle Memory-Runtime geworden.

Ich erkläre dir das System von oben nach unten, ohne in die einzelnen Codefunktionen abzusteigen.

1. Was ist V3?

Stell dir vor, du hast Tausende Dokumente, Notizen, Forschungsergebnisse und technische Dateien. Eine gewöhnliche Suchmaschine kann darin suchen. Ein Sprachmodell kann gefundene Texte interpretieren. Aber beides beantwortet allein noch nicht zuverlässig, welche Version eines Dokuments zu welchem Zeitpunkt vorlag, was sich seitdem geändert hat und auf welche konkrete Quelle sich eine Antwort stützt.

Genau dort setzt V3 an.

Die Pipeline beobachtet deine Dateien, vergibt nachvollziehbare Identitäten, erfasst ihre Zustände und hält die Entwicklung über aufeinanderfolgende Frames fest. Sie baut daraus einen strukturellen Speicher mit Textzugriff, Suche und Herkunftsnachweis.

Der entscheidende Unterschied ist: Das Gedächtnis speichert nicht nur, was es gefunden hat, sondern auch, zu welchem beobachteten Zustand und zu welcher Quelle dieser Fund gehört.

V3 ist damit ein erster ausführbarer Memory-Runtime-Kandidat. Ob sich daraus im praktischen Einsatz ein messbarer Vorteil gegenüber bestehenden Gedächtnissystemen ergibt, müssen wir noch prüfen.

2. Wie arbeiten die bisherigen K501-Bausteine zusammen?

1. Quelldaten

Deine Dateien bleiben die Quelle. Die Runtime beobachtet sie, statt sie umzuschreiben.

2. Pipeline in C

Scan, Objektidentität, Dateihash, Record und Änderungserkennung.

3. QH256 v2 + V3-Mapping

Explizite strukturelle Zustände pro Objekt; Zusammenführung gemäß MERGE_K.

4. Memory-Frame

Zustand, Textaufnahme, abgeleiteter Index, Gateway, Transition und Proof.

5. Zugriff und Rekonstruktion

Suchen, Herkunft verfolgen, Versionen vergleichen und frühere Frames prüfen.

Die Komponenten haben unterschiedliche Aufgaben.

Die C-Pipeline bildet den operativen Kern. Sie beobachtet den Dateibestand und erstellt die strukturellen Records.

QH256 v2 bleibt die kompakte Zustandsalgebra. Sie wird nicht neu definiert.

Der Indexer erzeugt aus dem geprüften Kandidaten eine abgeleitete Übersicht. Er wird nicht zum zweiten, unabhängigen Kanon.

Die Frame Engine steckt den jeweiligen Lauf in einen eindeutig benannten, zeitlich zugeordneten Zusammenhang. In V3 liegt diese Funktionalität innerhalb der Runtime; es handelt sich nicht um einen automatisch aufgerufenen separaten Dienst.

Das Gateway verbindet die Artefakte und macht den Speicher maschinell und menschlich navigierbar.

3. Was habe ich beim Mapping hinzugefügt?

Das ist eine der wichtigsten Neuerungen.

QH256 besitzt 128 Zellen mit jeweils zwei Bit. Jede Zelle kann `UNKNOWN`, `FALSE`, `TRUE` oder `GUARD` repräsentieren. Der ursprüngliche QH256-v2-Kern bleibt unverändert.

Neu ist das Anwendungsprofil `QH256-K501-AIONARC-MEMORY-V3A0`.

Es legt für die erste V3-Fassung die Bedeutung von 64 Zellen fest. Die übrigen 64 Zellen bleiben in diesem Profil zunächst `UNKNOWN` beziehungsweise reserviert.

Die zugeordneten Zellen sind in drei Bereiche gegliedert:

Zellen 0 bis 31: Objekt und Beobachtung. Hier werden beispielsweise Dateityp, beobachteter Pfad, Größeninformation, verfügbare Inhaltsprüfung, Record-Gültigkeit und bestimmte Eigenschaften der Textaufnahme repräsentiert.

Zellen 32 bis 47: Übergänge und Validierung. Diese Zellen bilden unter anderem ab, ob Objekte hinzugekommen sind, entfernt wurden oder sich verändert haben und ob zentrale Prüfungen der Kandidatenstruktur bestanden wurden.

Zellen 48 bis 63: Eigenschaften des beobachteten Korpus. Dazu gehören beispielsweise vorhandene Markdown- oder Textdateien, PDFs, Konfigurationsdateien, Quelldateien, Symlinks, leere Dateien und erkannte Duplikate von Inhalts-Hashes.

Damit wird QH256 erstmals in dieser Pipeline systematisch für das strukturelle Memory-Profil verwendet: sowohl für einzelne Objekte als auch für daraus zusammengefasste Zustände.

Eine wichtige Einschränkung bleibt bestehen. Die Zellen beschreiben nach festgelegten Regeln beobachtete Eigenschaften und Prüfergebnisse. Sie sind keine allgemeinen Wahrheitswerte über die Welt. Wenn `MERGE_K` mehrere Objekte zusammenführt, kann beispielsweise `GUARD` entstehen, weil unterschiedliche Objekte in derselben Zelle unterschiedliche Evidenz beitragen. Das bedeutet nicht automatisch einen logischen Widerspruch über ein einzelnes Objekt.

Status: Das V3-Mapping ist spezifiziert und im Paket implementiert, aber noch ein Kandidat zur Prüfung. Erst nach deiner Prüfung, weiteren Tests und einer ausdrücklichen Freigabe kann es als kanonisches K501-Anwendungsprofil übernommen werden.

4. Was bedeutet Proof Before State in V3?

Hier wurde aus einem Grundprinzip ein konkreter operativer Vertrag.

Bevor ein neuer Frame als abgeschlossen gilt, erstellt die Runtime zunächst einen Kandidaten. Anschließend prüft sie unter anderem die Objektidentitäten, Record-Hashes, gespeicherten Zustände, Textbindungen, Indexeinträge, Übergänge, Gateway-Referenzen und QH256-Daten.

Der Proof wird geschrieben und gesichert, bevor der Kandidatenzustand als `STATE.tsv` in den endgültigen Frame übernommen wird. Erst wenn die definierten Prüfungen bestanden sind, kann dieser Commit erfolgen.

Bei einem Fehler darf die Runtime keinen scheinbar gültigen Frame präsentieren. Ein unvollständiges Staging-Verzeichnis ist kein erfolgreicher Commit und wird nicht automatisch übernommen.

Das ist die praktische Bedeutung von Proof Before State in dieser Implementierung.

Es gibt jedoch eine klare Grenze: Der Proof dokumentiert, dass die benannten lokalen Prüfungen bestanden wurden. Er ist keine digitale Signatur, kein Schutz gegen einen Angreifer, der den gesamten Speicher kontrolliert und neu berechnet, und kein Beweis dafür, dass der Inhalt eines Dokuments wahr ist.

Ein `PASS` muss deshalb immer zusammen mit dem geprüften Zustand, dem Vertrag und den Grenzen der Prüfung gelesen werden.

5. Was kannst du damit praktisch machen?

V3 ist nicht nur auf das Erzeugen von Hashes ausgelegt. Die aktuelle Fassung erlaubt dir, aufgenommene Texte mit einer exakten lexikalischen Suche zu durchsuchen und einen Treffer zu seiner Objektidentität und Quelle zurückzuverfolgen.

Du kannst damit zum Beispiel feststellen, welche Datei einen bestimmten Begriff enthält, welchem Frame sie zugeordnet ist, was sich gegenüber dem vorherigen Frame verändert hat und ob die ursprüngliche Quelle noch mit dem gespeicherten Zustand übereinstimmt.

Änderungen werden als neue Frames erfasst; frühere Frames werden nicht stillschweigend überschrieben. Wenn mehrere mögliche Vorgänger beziehungsweise Branch-Enden existieren, darf die Runtime nicht einfach einen davon als den aktuellen auswählen.

Damit entsteht ein strukturierter, historisch nachvollziehbarer Speicher mit überprüfbarem Quellenbezug.

Aber ich möchte auch hier präzise bleiben: Die Suche ist derzeit eine exakte lexikalische Suche. Sie versteht keine Bedeutungsähnlichkeit, erzeugt keine intelligenten Zusammenfassungen und beantwortet keine Fragen mit eigener sprachlicher Interpretation. Embeddings, semantische Suche und llama.cpp sind bewusst noch nicht integriert.

6. Was wurde tatsächlich geprüft?

Die Paketversion wurde in der hier verfügbaren Build-Umgebung kompiliert und getestet. Die dokumentierte End-to-End-Suite umfasst unter anderem die QH256-Kernprüfungen, einen initialen Frame, einen Folgeframe, die Vorgängerkette, Suche und Trace, Diff und History, veränderte Quelldaten, manipulierte Indexdaten, wiederverwendete Frame-IDs sowie mehrdeutige Branch-Enden.

Die Tests bestanden. Auch die Paketprüfsummen und die mitgelieferte v2.1-C-Referenzarchiv-Prüfsumme wurden geprüft. Ein Sanitizer-Lauf fand zunächst einen Speicherbereinigungsfehler; dieser wurde behoben, danach bestand die Suite erneut.

Diese Ergebnisse bedeuten: Die getestete V3-Paketfassung besteht die in ihr definierten lokalen Prüfungen. Sie bedeuten nicht, dass sie bereits auf deinem HP-xw8600-Rechner validiert wurde, bei 500 oder 50.000 Dateien zuverlässig skaliert oder unabhängig geprüft ist. Diese Gates bleiben offen.

7. Der weitere Weg

Du liest zunächst alle Dokumente in Ruhe. Danach führen wir die Pipeline kontrolliert lokal aus. Wir beginnen mit einer Bestandsaufnahme und einem kleinen, getrennten Testkorpus, prüfen Quellcode und Vertrag, bauen mit dem lokalen Compiler, führen die Tests aus und vergleichen die Ergebnisse mit dem erwarteten Verhalten.

Danach erst entscheiden wir über die Freigabe des Mappings, die Lizenzlage des übernommenen v2.1-C-Codes, die Release-Gates und den Git-Ablauf. Freeze, Commit und Push nach GitHub erfolgen erst, wenn die zugehörigen Prüfungen bestanden sind und du die entsprechende Freigabe erteilst.

Die Fassung, die du jetzt vorliegen hast, bleibt bis dahin ein nicht-kanonischer Release-Kandidat.

Abschließende Einordnung.

V3 ist der Versuch, aus den bereits vorhandenen K501-Bausteinen einen möglichst kleinen, verständlichen und bedienbaren strukturellen Gedächtniskern zu machen. Neu sind nicht einfach zusätzliche Funktionen, sondern die explizite Verbindung von Objektbeobachtung, QH256-Anwendungsprofil, Frame-Historie, abgeleiteten Textdaten, Provenienz und einem konkreten Proof-Before-State-Vertrag in einer ausführbaren C-Runtime.

Das eigentliche Ziel ist, dass du später deine eigenen Daten aufnehmen und nachweisen kannst, wie der gespeicherte Informationszustand entstanden ist, was sich geändert hat und woher ein Suchergebnis stammt. Erst auf dieser Grundlage soll ein Sprachmodell als getrennte Interpretationsschicht hinzukommen.

K501 GO bleibt bestätigt — für die kontrollierte Prüfung und Weiterentwicklung von V3, nicht für eine vorweggenommene kanonische Freigabe.